DATABASE ARCHITECTURE EVOLUTION

DeskFlow — Migration Report

Transitioning from MongoDB/Mongoose to PostgreSQL/Prisma

Backend System Upgrade • Frontend Contract Unchanged

Core Migration Strategy

- **Relational Data Modeling**
  Transition loosely-typed Mongoose ObjectIds into strictly enforced Postgres tables with full foreign key constraints.
- **Flattened Data Layouts**
  Mongoose embedded sub-documents (like `aiAnalysis`) flattened directly into primitive columns.
- **Explicit App Logic**
  Replaced implicit schema hooks (hashing, validation) with cleaner, explicit application controllers & utility formatters.
- **SQL Parameterization**
  Removed `express-mongo-sanitize`. Parameterized queries in Prisma provide native protection against SQL injection.
- **Typed Error Handling**
  Swapped string-based Mongoose error parsing to explicit, typed database codes (e.g., `P2002`).
- **ID Modernization**
  Upgraded legacy MongoDB `_id` fields into globally standardized `UUID` primary keys.

Streamlined Architecture Blueprint

Eliminated `src/models/` completely. The entire schema tier is now self-contained inside the schema definition layer.

```
backend/
├── prisma/
│   ├── schema.prisma      # Replaces User.js + Ticket.js mode
│   ├── seed.js            # New relational data seeder
│   └── migrations/        # Safe versioned migration tracks
├── src/
│   ├── config/prisma.js   # Single client provider instance
│   ├── controllers/       # Refactored for client execution
│   ├── services/          # Business workflows updated
│   └── utils/formatUser.js# Explicit payload stripping
```

What Changed?

A streamlined structure leveraging standard migrations instead of volatile ad-hoc startup scripts.

What Remained?

Route signatures, API payload interfaces, controller authorization profiles, and third-party AI integrations like Groq Service.

The Relational Schema Engine

```
model User {
  id        String   @id @default(uuid())
  email     String   @unique
  password  String
  role      Role     @default(employee)
  tickets   Ticket[]

  @@map("users")
}

enum Role {
  admin
  employee
}
```

```
model Ticket {
  id         String   @id @default(uuid())
  title      String
  status     String   @default("Open")
  priority   Priority @default(Medium)

  // Flattened AI fields
  aiSummary  String?

  createdById String
  createdBy  User @relation(fields:..., onDelete: Cascade)
}
```

Key Schema Design Decisions

1. Enums vs App Validation

Priority utilizes real Postgres native enums. However, **Status** and **Category** leverage `String` fields coupled with application-tier validation to keep API strings formatting clean and uniform (avoiding space mapping issues).

2. Cascading Referential Integrity

Configured `onDelete: Cascade` rules natively inside Postgres. Eliminates database-side anomalies and guarantees orphaned child records are cleaned up safely during profile actions.

3. Non-Guessable Identifiers

Leverages standard `UUID v4` primary keys rather than standard incremental integers. Obfuscates resource volume sizing metrics and blocks malicious ID iteration attempts.

Mongoose vs. Prisma: API Translator

BEFORE: Mongoose (NoSQL Approach)

```
// Complex sorting and document decoration
const tickets = await Ticket.find(filter)
  .populate('createdBy', 'name email')
  .sort({ createdAt: -1 });
```

```
// Dynamic text indices lookups
filter.$text = { $search: search };
```

AFTER: Prisma + Postgres (Strict SQL)

```
const tickets = await prisma.ticket.findMany({
  where: filter,
  include: { createdBy: { select: { name: true } } },
  orderBy: { createdAt: 'desc' },
});
```

```
where.OR = [
  { title: { contains: search, mode: 'insensitive' } }
];
```

Postgres-Enabled Roadmap Improvements

Native Enterprise Full-Text Search

Move away from `ILIKE` query matchers to optimized `tsvector` lexemes index strategies. Drastically improves speed, weights ranking matching, and scoring scaling capacities.

High-Performance Analytics

Leverage relational group operations natively via `GROUP BY` optimizations to query and map live trend lines on Ticket matrices directly on the raw system index layer.

Deterministic Guarding

Hardened data consistency boundaries. Invalid combinations are rejected immediately at the base kernel storage standard, safe-guarding operational business logic integrity.